

Implementation Breakdown: Recursive Asymptotic Variance Estimation for DTMCs

MS Thesis Technical Documentation

September 2026

1 Introduction

This document provides a technical explanation of the Python implementation (`src/simulate_dtmc.py`) of the recursive stochastic approximation (SA) algorithm for estimating the asymptotic variance $\kappa(f)$ of a function f defined on a Discrete-Time Markov Chain (DTMC).

The implementation is based on the research paper: "*Markov Chain Variance Estimation: A Stochastic Approximation Approach*" (Agrawal et al., 2024).

2 Mathematical Foundations

Given an irreducible and aperiodic DTMC on a finite state space $\mathcal{S}$ with transition matrix P and stationary distribution π , we define the stationary expectation of a bounded function $f : \mathcal{S} \rightarrow \mathbb{R}$ as $\bar{f} = \mathbb{E}_\pi[f(X)]$.

2.1 Asymptotic Variance $\kappa(f)$

The asymptotic variance is defined via the Central Limit Theorem for Markov Chains:

$$\frac{\sum_{k=0}^{n-1} f(X_k) - n\bar{f}}{\sqrt{n}} \implies \mathcal{N}(0, \kappa(f)) \quad \text{as } n \rightarrow \infty \quad (1)$$

The implementation uses a formulation involving the solution V to the **Poisson Equation**:

$$f - \bar{f}\mathbf{1} = V - PV \quad (2)$$

Rearranging the terms, the script evaluates $\kappa(f)$ using the identity:

$$\kappa(f) = \mathbb{E}_\pi[2f(X)V(X) - 2f(X)\bar{V} - f^2(X) + f(X)\bar{f}] \quad (3)$$

where $\bar{V} = \mathbb{E}_\pi[V(X)]$.

3 Algorithm Implementation

The core of the simulation is the `TabularVarianceEstimator` class, which implements **Algorithm 1** from the paper.

3.1 The Update Logic

At each transition (X_k, X_{k+1}) , the estimator performs four recursive $O(1)$ updates using a step-size α_k :

1. **Stationary Mean** ($\bar{f}_k$): Updates the estimate of $\mathbb{E}_\pi[f]$.
2. **Value Function** (V_k): Solves the Poisson equation. The code uses a compact vector update that preserves the orthogonality constraint $\mathbf{1}^T V_k = 0$.
3. **Stationary Mean of V** ($\bar{V}_k$): Tracks the average of the value function.
4. **Asymptotic Variance** (κ_k): Aggregates the terms from the $\kappa(f)$ identity using linear SA.

4 Verification and Invariants

To ensure the simulation is correct, the implementation includes several verification mechanisms:

4.1 Analytical Ground Truth

The script includes a `compute_analytical_ground_truth` function that solves the system exactly using matrix inversion and least squares. It cross-validates the result using two independent formulations of $\kappa(f)$:

1. $\kappa(f) = 2\mathbb{E}_\pi[(f(X) - \bar{f})V(X)] - \mathbb{E}_\pi[(f(X) - \bar{f})^2]$
2. $\kappa(f) = \mathbb{E}_\pi[V^2(X)] - \mathbb{E}_\pi[(PV)^2(X)]$

4.2 Spectral Gap and Convergence Constants

The convergence of the algorithm depends on the spectral gap Δ_1 of the symmetric part of $D_\pi(I - P)$. The function `compute_theoretical_constants` uses this gap to derive scale factors c_1, c_2, c_3 that satisfy the stability conditions derived in Theorem 2.1 of the paper.

4.3 Unit Testing

The `src/test_simulate_dtmc.py` suite verifies the **orthogonal conservation invariant**. It mathematically proves that for any update V_{k+1} , the property $\sum_{i \in \mathcal{S}} V_{k+1}(i) = 0$ is preserved exactly (within machine precision) without requiring an explicit projection step at every iteration.

5 Usage and Visualization

The simulation generates a convergence plot (`src/convergence_plot.png`) comparing diminishing step-sizes (guaranteed $O(1/n)$ convergence) against constant step-sizes (faster initial descent but higher asymptotic bias).